



POD-Attention: Unlocking Full Prefill-Decode Overlap for Faster LLM Inference

Aditya K Kamath¹, Ramya Prabhu², Jayashree Mohan²,
Simon Peter¹, Ramachandran Ramjee², Ashish Panwar²



**¹University of
Washington**



**²Microsoft Research
India**



Outline

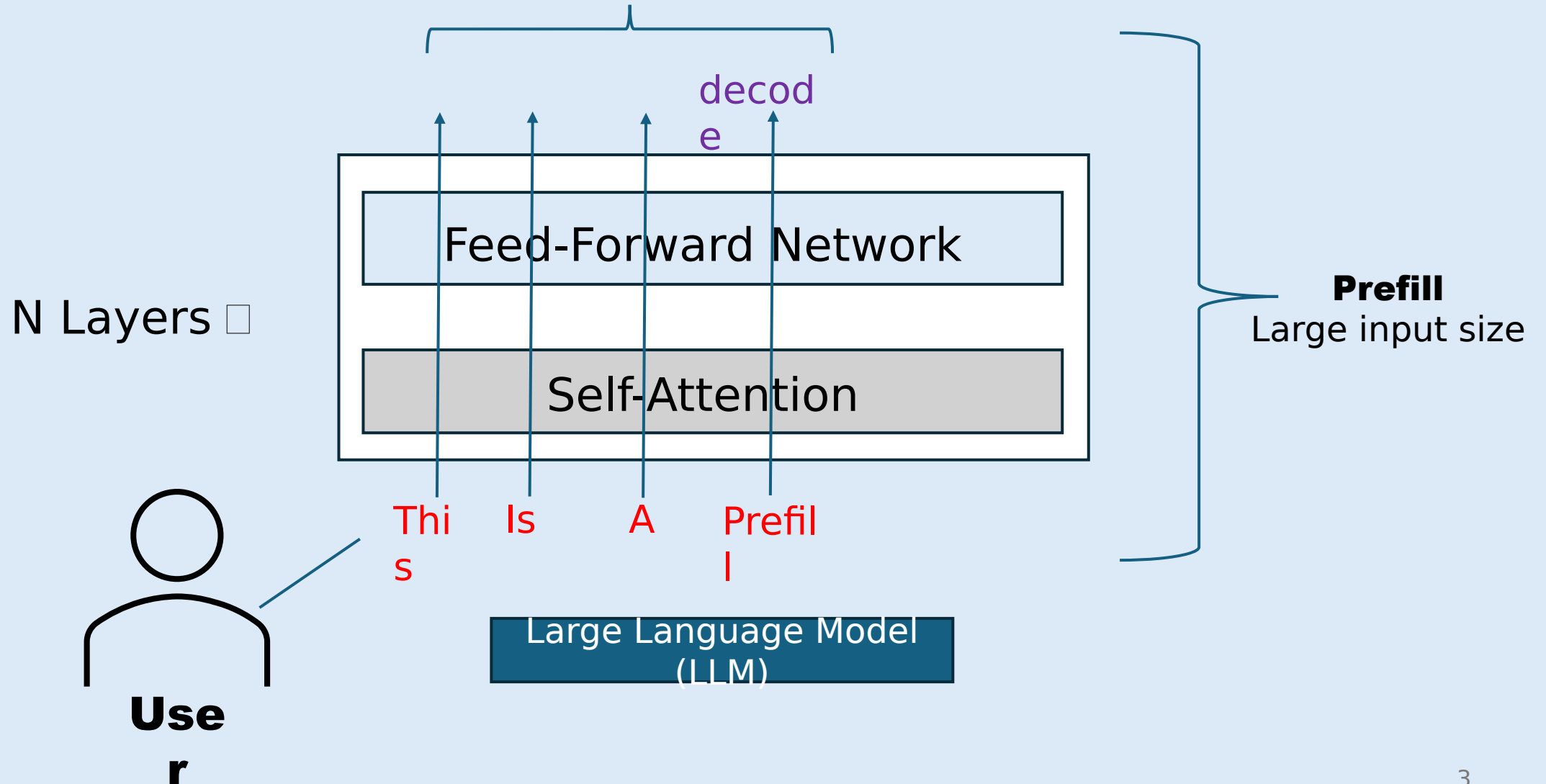
- LLM Inference and Hybrid Batching
- Motivation for POD-Attention
- GPU microarchitectural background and kernel fusion
- POD-Attention design
- Evaluation



LLM Prefill

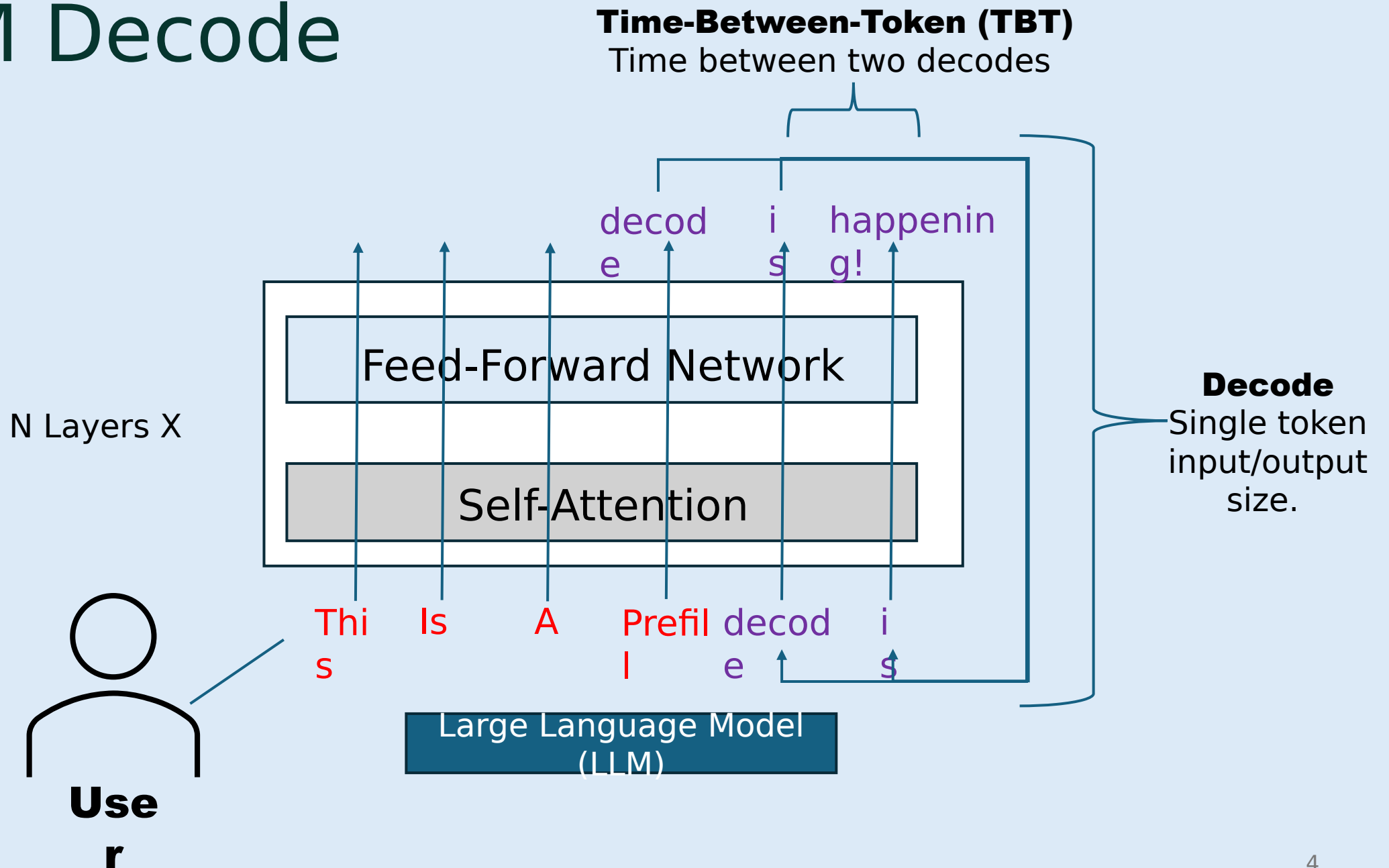
Time-To-First-Token (TTFT)

Time taken by this iteration



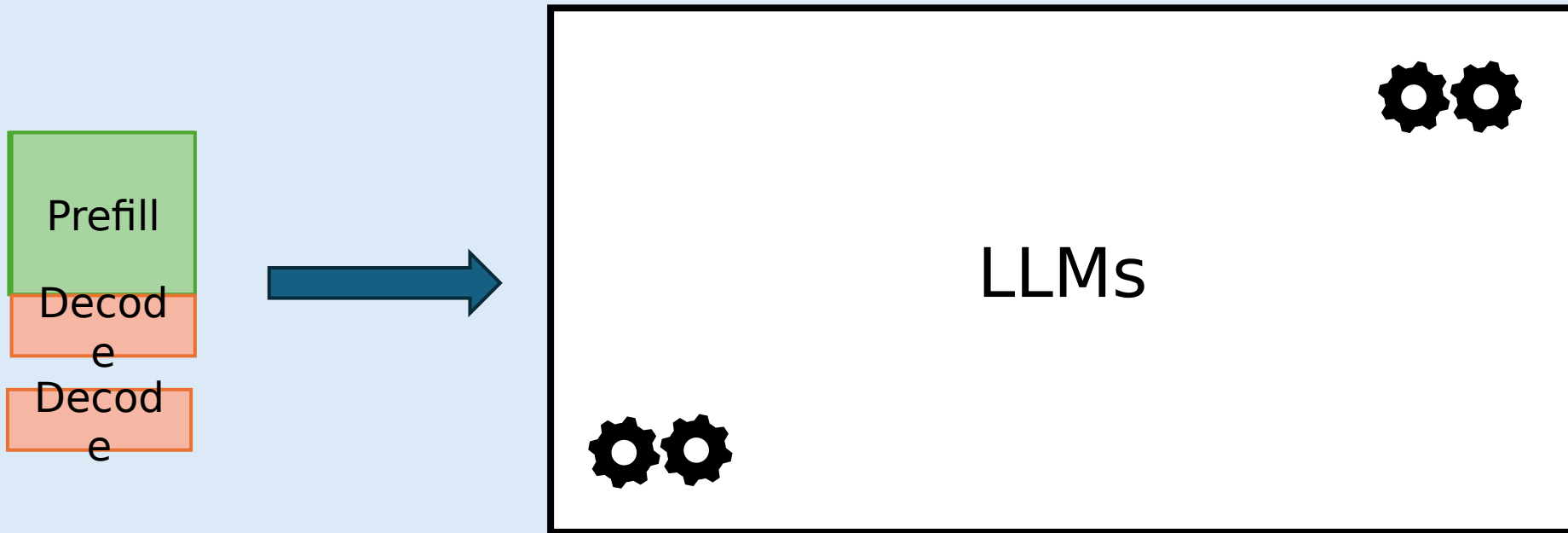


LLM Decode





LLM Hybrid Batching [Sarathi-Serve, OSDI'24]

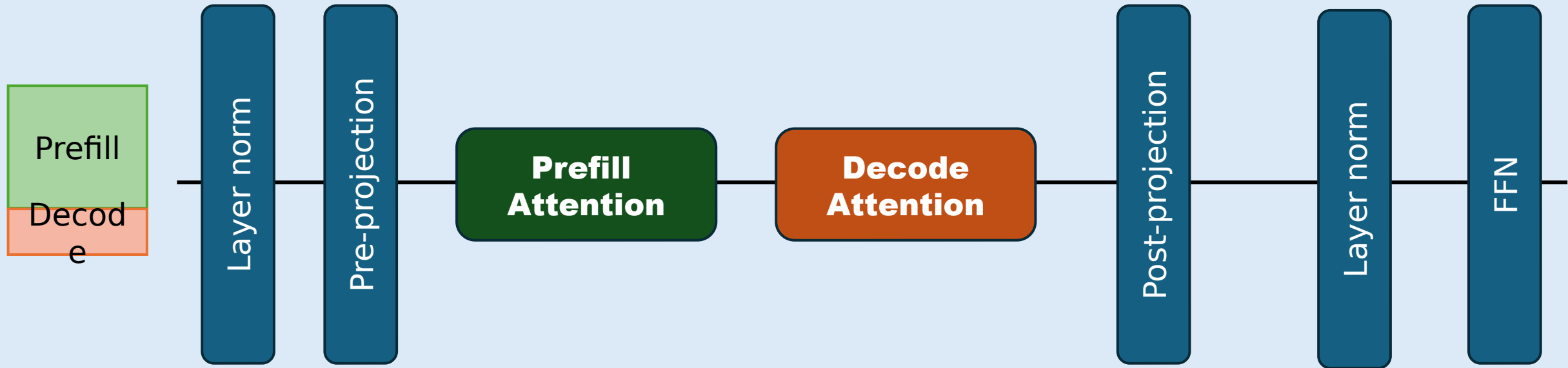


Prefill and decode inputs of multiple requests are combined as a single input
Improves throughput by reusing model weights and reducing latencies.



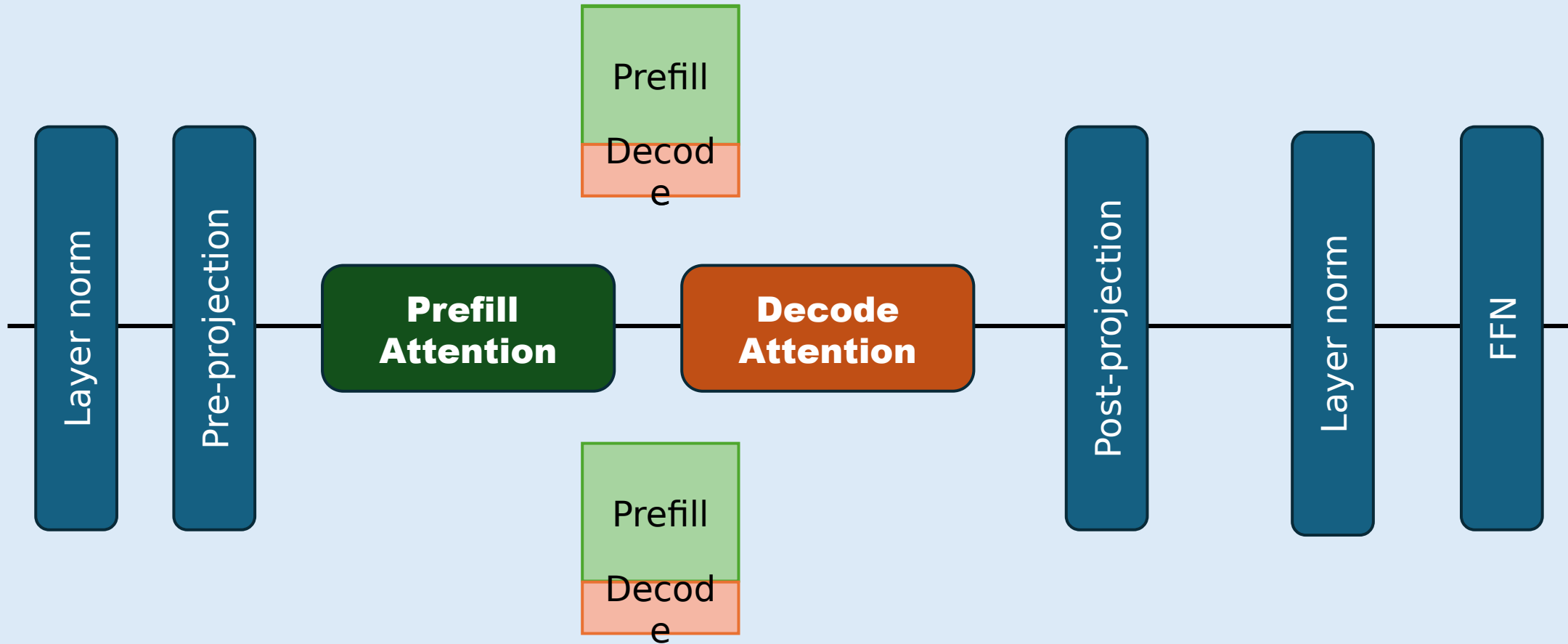


LLM Hybrid Batching – A Closer Look



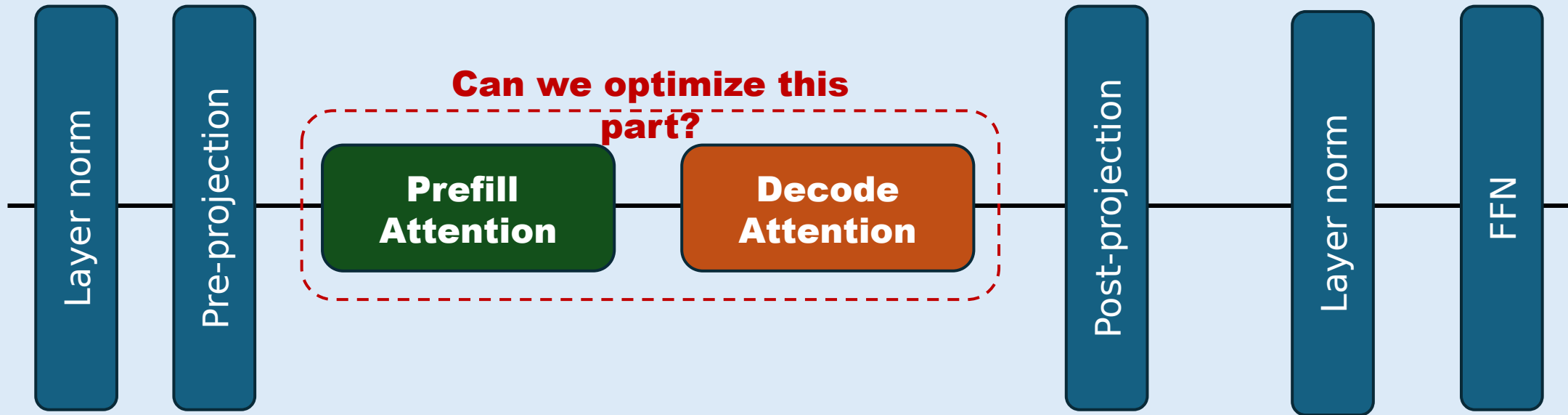


LLM Hybrid Batching – A Closer Look



**Attention operates on discrete data items (no data reuse across input requests).
Kernels optimized independently for prefill or decode.**

Our Goal With POD-Attention





Why Optimize Attention?

**60% of overall
time**

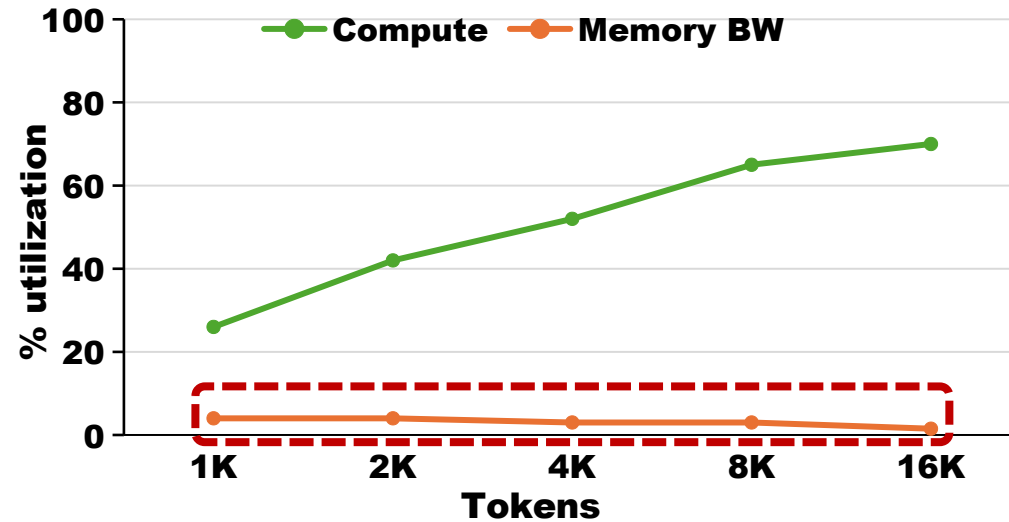
Model: Llama-3-8B, Hardware A100 x 2, Decode Batch Size: 60

Attention is the biggest bottleneck at large context lengths.

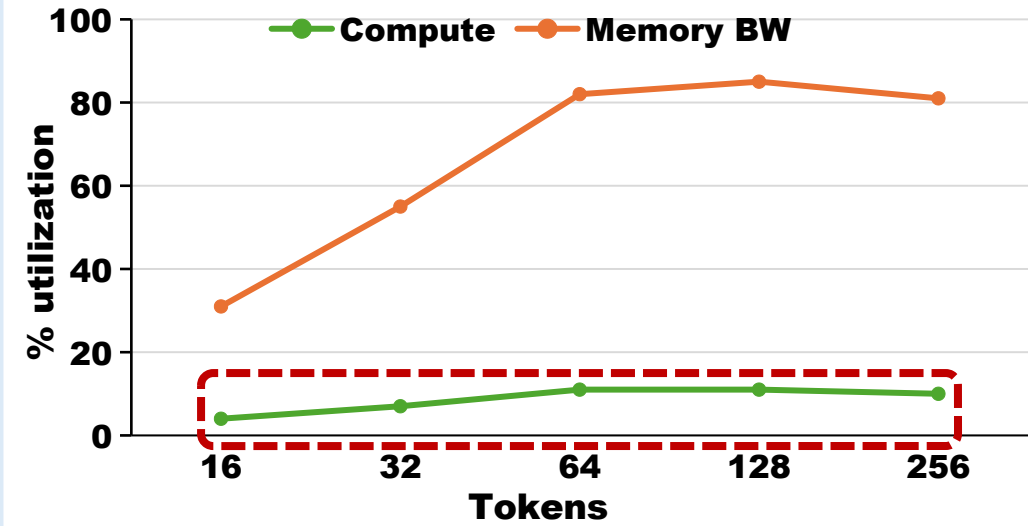


Opportunity: Resource (Under)utilization

Prefill attention



Decode attention

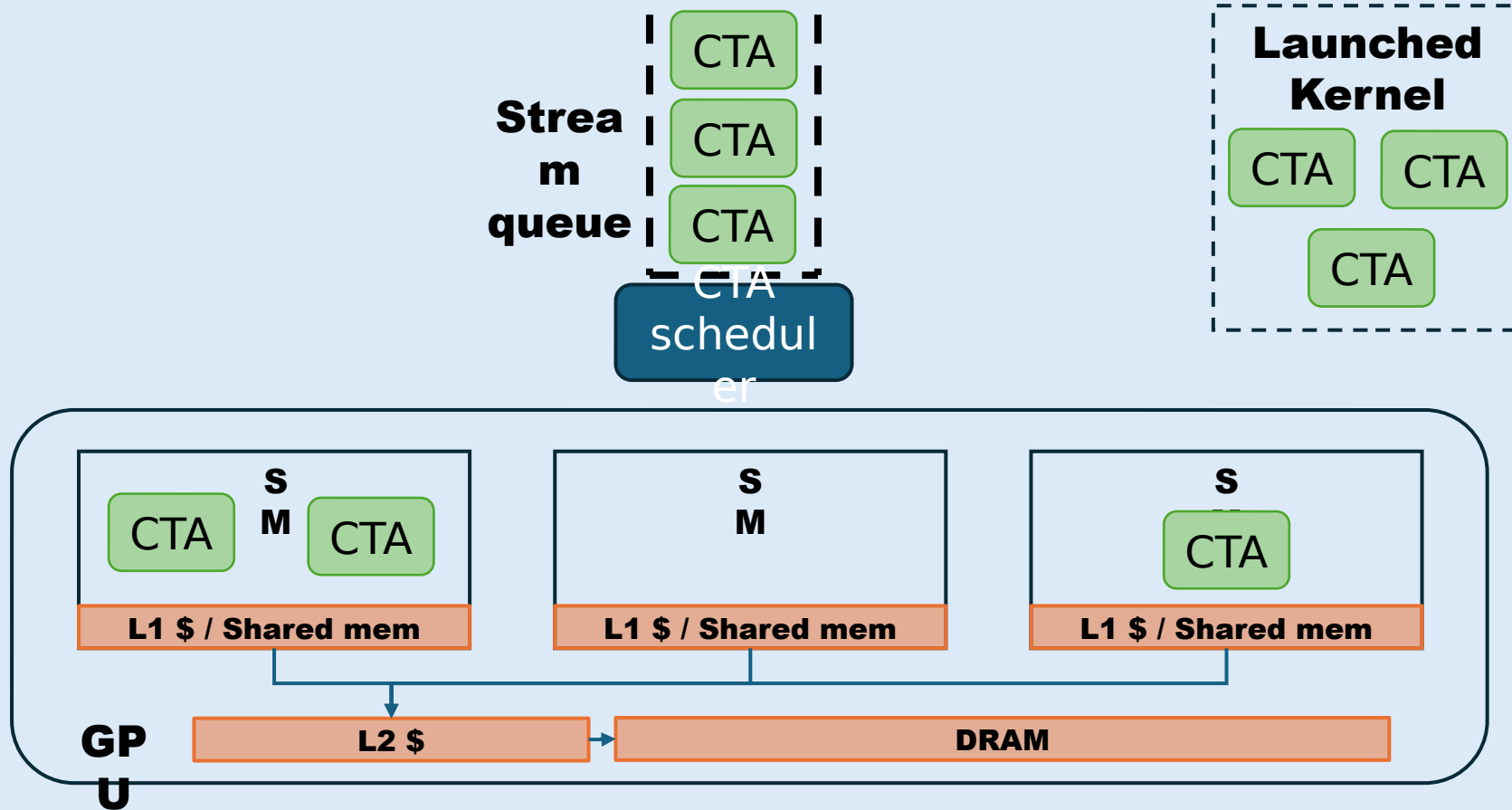


Model: Llama-3-8B, Hardware: 2
A100

Prefill saturates compute but has low memory bandwidth utilization.
Decode saturates memory bandwidth but has low compute utilization.



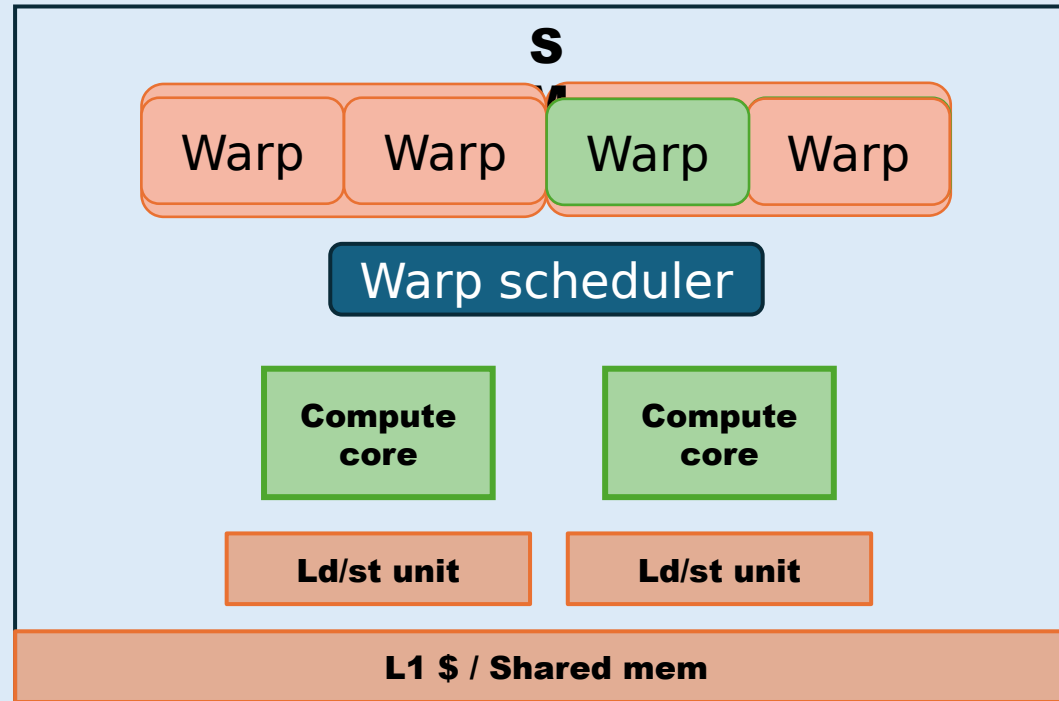
GPU Execution: CTA Scheduling



CTA scheduler in hardware assigns queued CTAs to SMs for execution.



GPU Execution: Warp Scheduling



**Green = Prefill
Warp**
**Orange = Decode
Warp**

Warp scheduler assigns queued warps to free resources attempting to saturate all GPU resources.



Fusing Prefill and Decode

- **Goal:** Overlap compute-heavy prefill with memory-bandwidth-heavy decode to fully utilize GPU resources.
 - **Sub Goal:** Co locate prefill and decode CTAs within the same SM

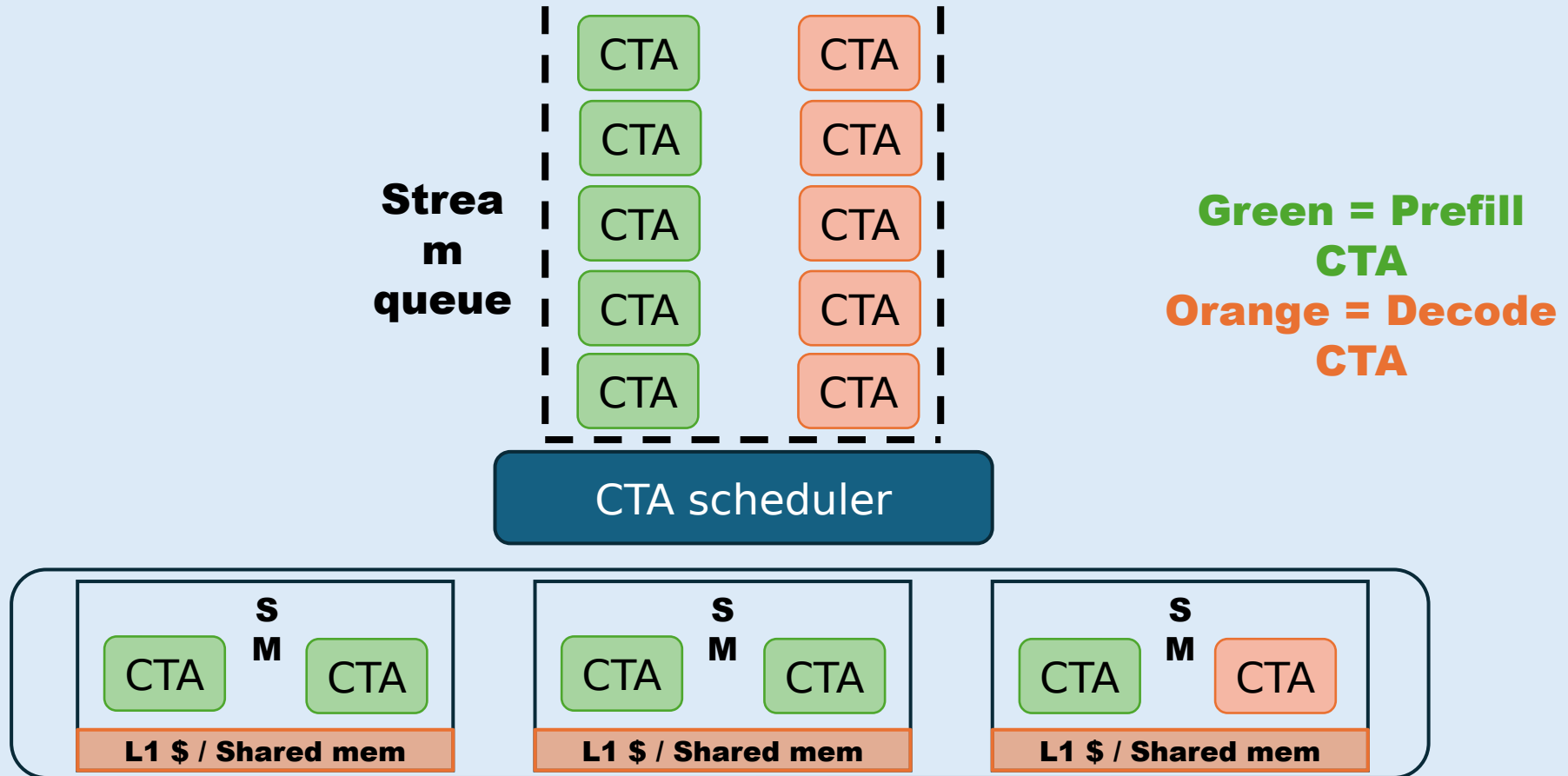


Known Methods of Kernel Fusion

- **Naïve fusion (CTA-parallel):** Combined kernel for both prefill and decode.
- **Kernel-parallel:** Launch kernels in separate CUDA Streams.
Cannot guarantee operation co-location within SM.
- **Warp-parallel:** Divide warps in the CTA among ops [HFuse, CGO '22].
Exhibits bad load balancing within the GPU.
- **Intra-thread:** Every thread performs both operations.
Excessive synchronization overhead.



Naïve Kernel Fusion (CTA-Parallel)



Cannot guarantee SM co-location, limiting speedup.



How Can We Address This?

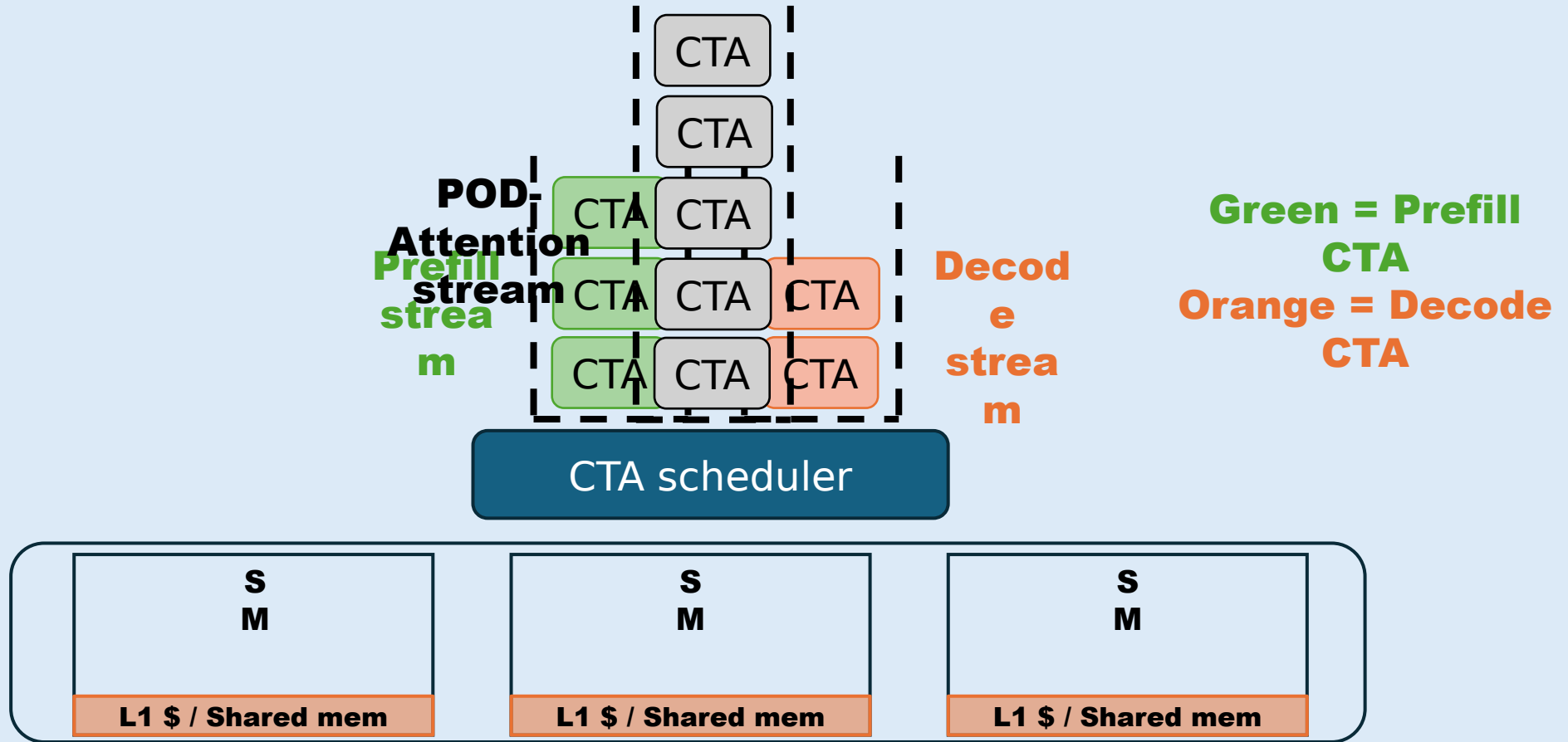
- **Goal:** Overlap compute-heavy prefill with memory-bandwidth-heavy decode to fully utilize GPU resources.
- **Problem:** CTA scheduler is handled by GPU driver/hardware.



Introducing POD-Attention!

- **POD-Attention:** Combines prefills and decodes into a single kernel with guaranteed SM co-location.
- Key idea: **SM-aware CTA scheduling**
 - Guarantees each SM runs prefill and decode CTAs in parallel.
 - Enables the CTA scheduler to overlap the two operations.
 - Utilizes compute and memory bandwidth simultaneously.

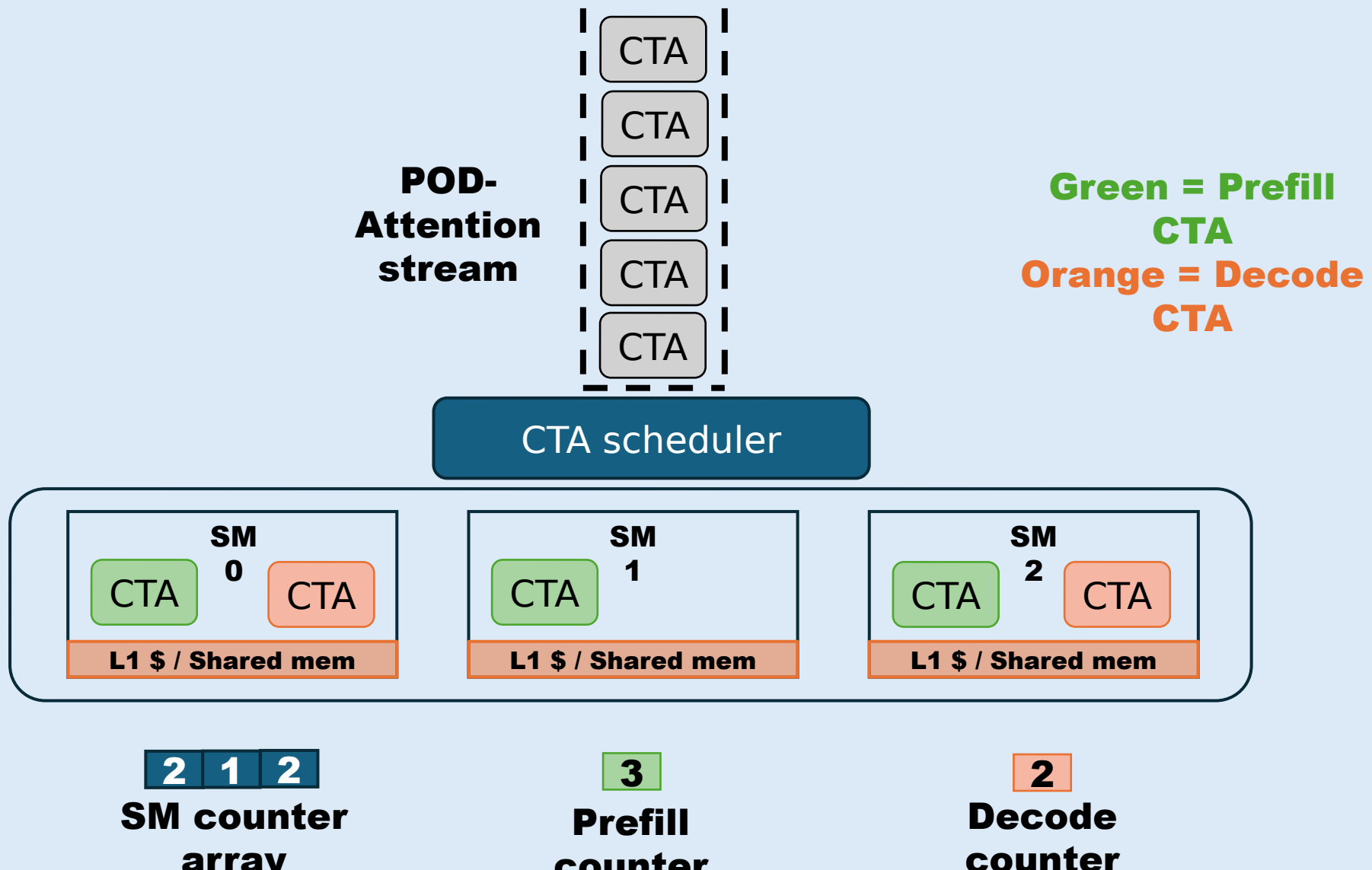
POD-Attention: Fused kernels



We create a new kernel with enough CTAs for both prefill and decode kernels.

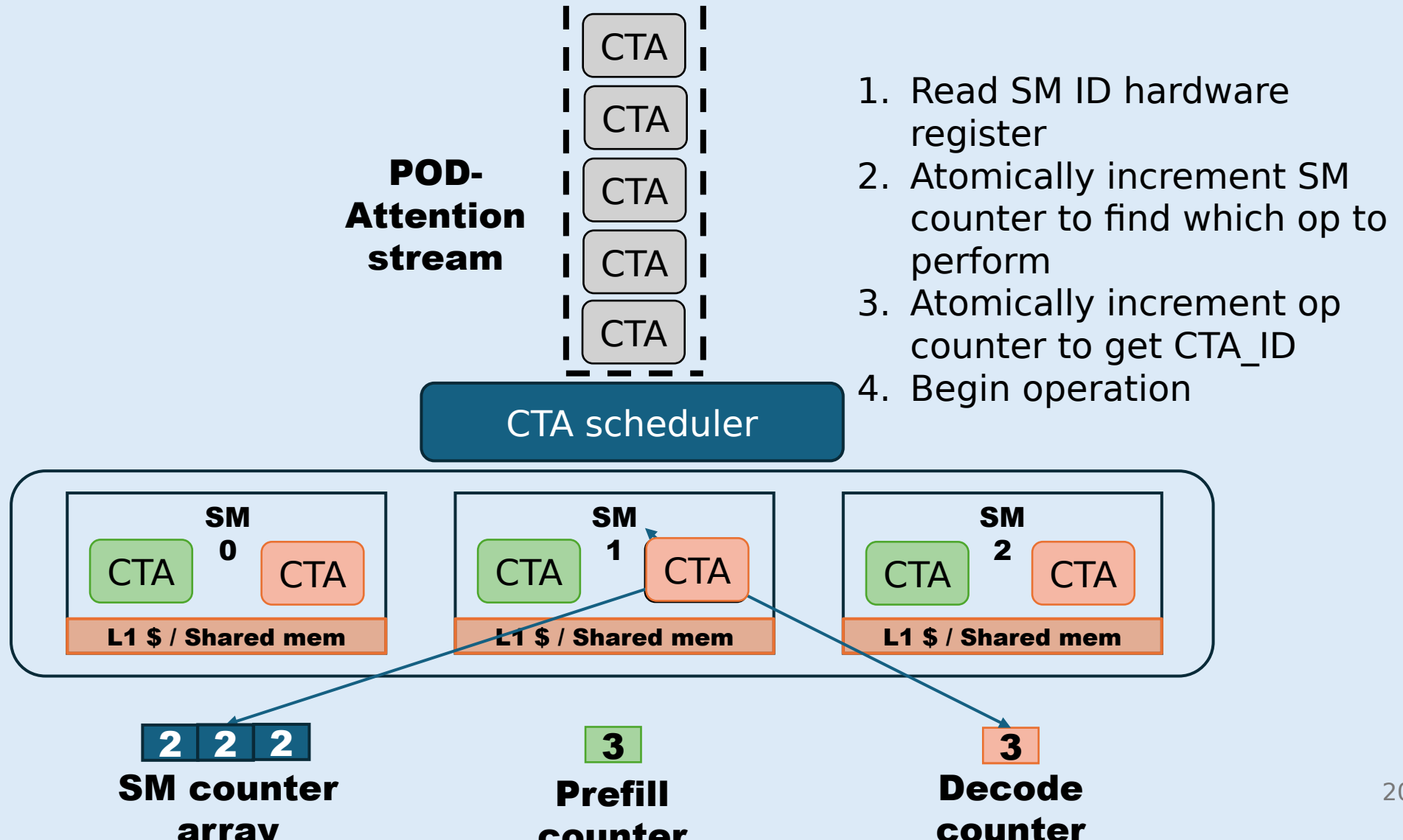


POD-Attention: SM-aware CTA scheduling





POD-Attention: SM-aware CTA scheduling

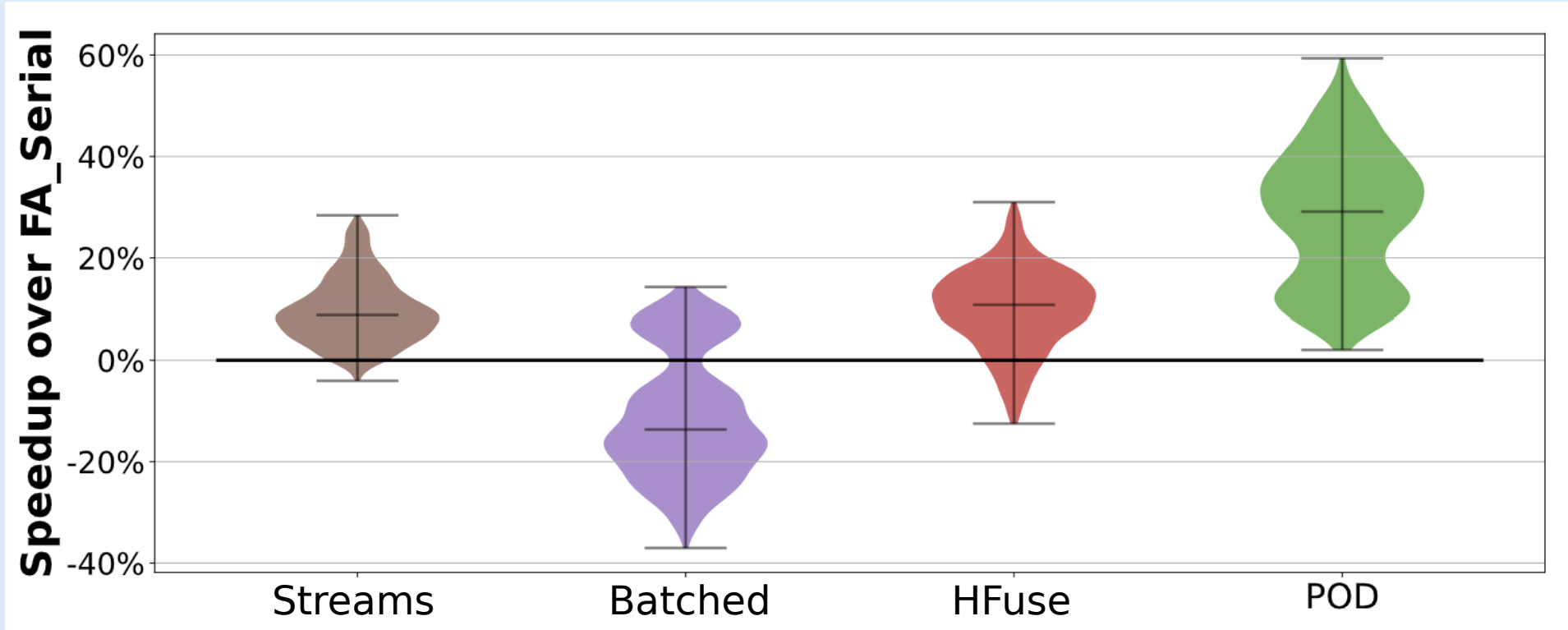




Attention-specific Optimizations

- Decode tiling
 - Smaller tiles minimize compute utilization of decode CTAs
- Varying number of CTAs per SM
 - Enables fine-grained overlap of prefill and decode CTAs
- Limited prefill splitting
 - Reduces memory bandwidth usage of prefill CTAs

POD-Attention Performance



POD beats all existing alternatives, with up to 61% speedup (33% average).



LLM Evaluation: Baselines and Workloads

Inference frameworks

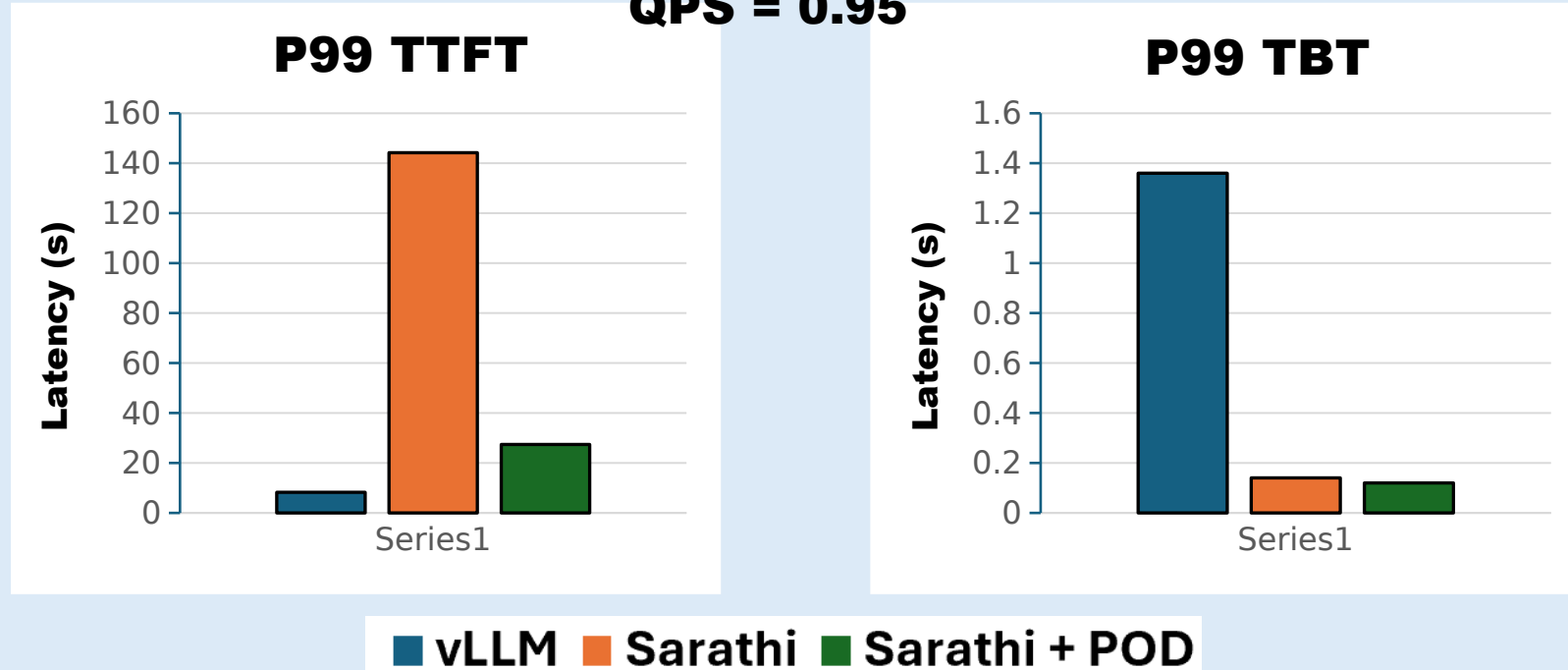
Framework	Description
vLLM [SOSP '23]	Popular framework that prioritizes prefill latency (TTFT).
Sarathi [OSDI '24]	Uses hybrid-batching to improve decode latency (TBT).
Sarathi + POD	Sarathi enhanced with POD-Attention.

Models and hardware

Model	GPU	# KV Heads	# Layers
Yi-6B	1 A100	32	4
Llama-2-7B	2 A100s	32	32
Llama-3-8B	2 A100s	32	8

Online Inference Performance

**Llama 3 8B, arXiv-Summarization,
QPS = 0.95**



- vLLM provides low TTFT but **high TBT**, stalling nearly all requests
- Sarathi provides low TBT, but **high TTFT**
- Sarathi + POD reduces TBT stalls with TTFT closer to vLLM



Summary



- POD-Attention fuses prefill and decode attention to maximize GPU resource utilization
- Performance improvement:
 - Up to 61% faster hybrid attention over current methods
 - Up to 22% higher end-to-end throughput
 - Lower latency and stalls than state-of-the-art frameworks vLLM and Sarathi
- Available for FlashAttention:
https://github.com/microsoft/vattention/tree/main/pod_attn
- As well as FlashInfer:
<https://github.com/flashinfer-ai/flashinfer>



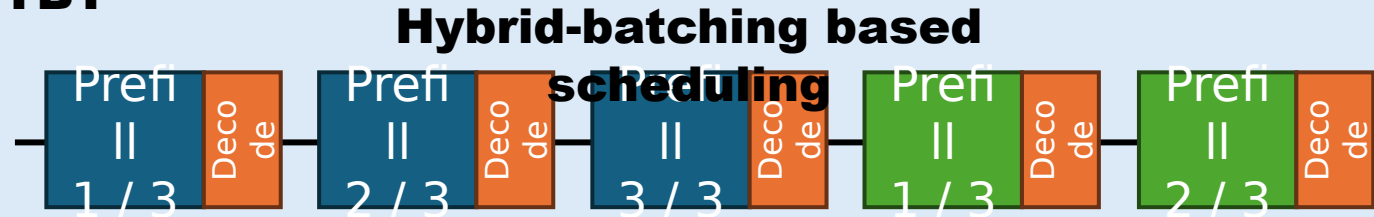
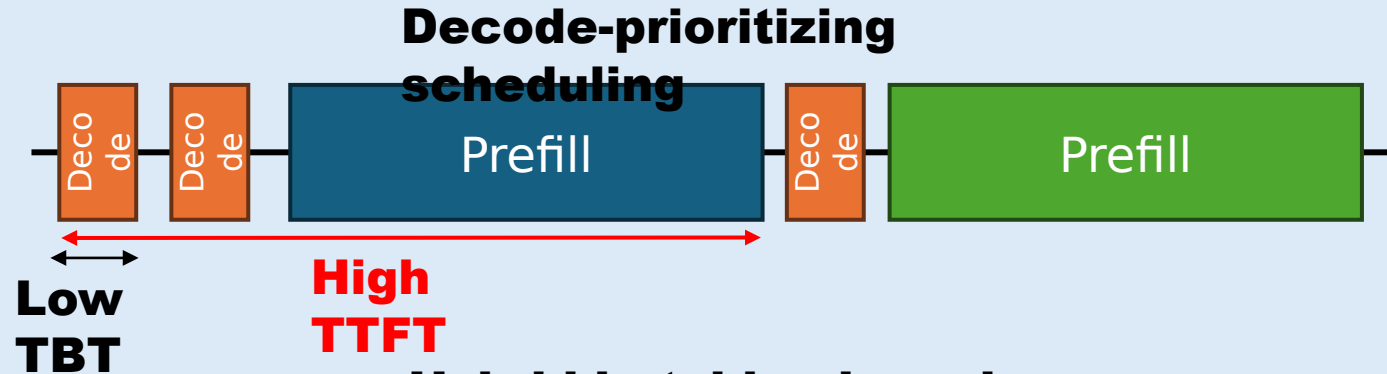
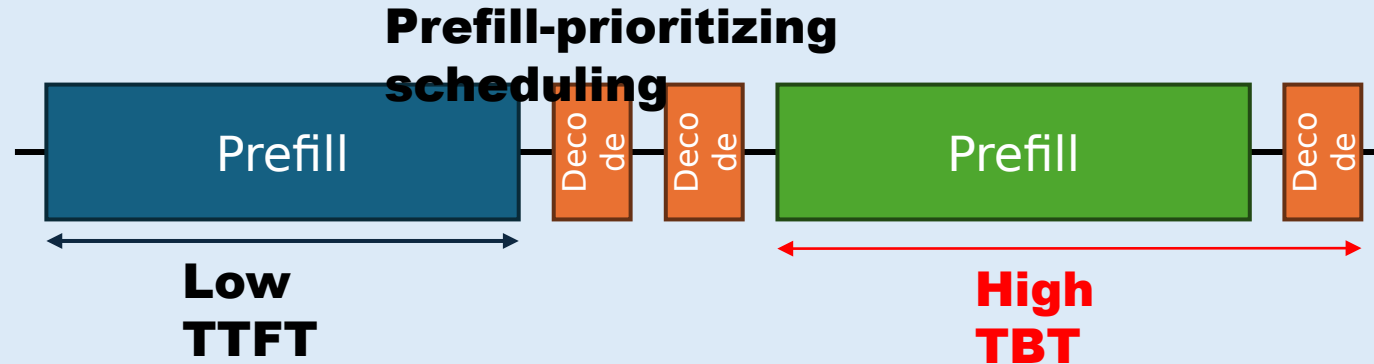
Thank you



Backup slides



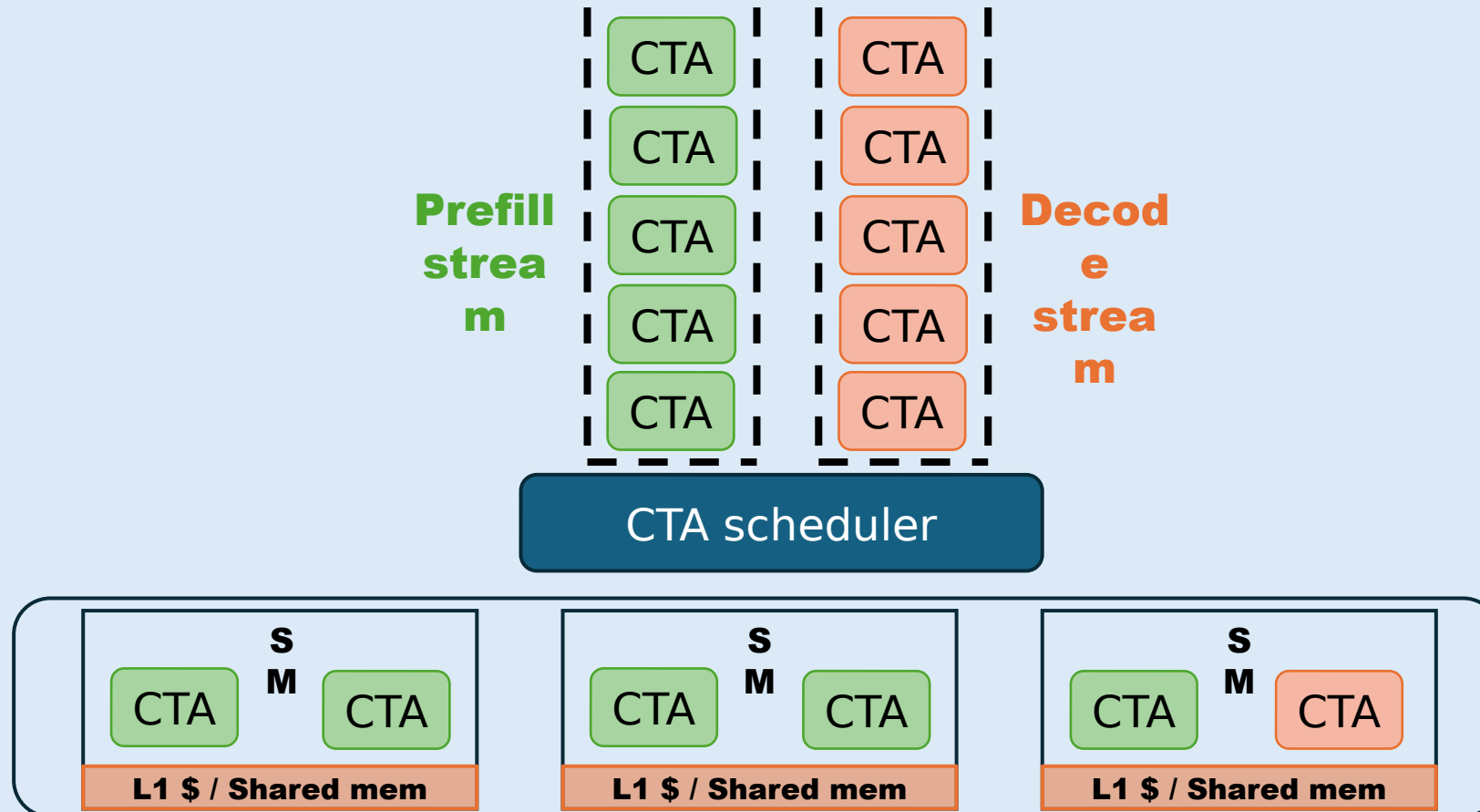
LLM Scheduling



Hybrid-batching trades off slightly higher TTFT for low TBT



Kernel-parallel Execution with Streams



Streams cannot guarantee SM co-location, limiting potential speedup.



Offline Inference Performance

Up to 27% higher throughput than Sarathi and vLLM



Online Inference Performance

QPS	System	TTFT		TBT		Request Latency		% Requests with Stalls	
		P50	P99	P50	P99	P50	P99	200ms	500ms
1.1	vLLM (original)	0.67	10.11	0.04	1.13	25.05	91.01	99.95	97.8
	Sarathi	2.2	12.58	0.10	0.15	26.83	92.24	2.05	0
	Sarathi+POD	1.9	12.26	0.10	0.14	24.70	79.04	3.17	0
1.2	vLLM (original)	0.94	12.70	0.07	1.76	42.73	151.8	99.95	99.6
	Sarathi	25.44	57.83	0.12	0.16	67.12	140.5	5.07	2.63
	Sarathi+POD	7.49	23.78	0.11	0.15	38.69	106.8	2.29	0

Table 5. Internal workload. Latency numbers in seconds.

QPS	System	TTFT		TBT		Request Latency		% Requests with Stalls	
		P50	P99	P50	P99	P50	P99	200ms	500ms
0.85	vLLM (original)	0.55	6.26	0.03	0.82	20.53	234.93	99.9	97.8
	Sarathi	2.68	14.89	0.08	0.13	27.87	281.07	4.15	2.05
	Sarathi+POD	1.85	12.71	0.08	0.11	24.31	255.75	1.85	1.61
0.95	vLLM	0.71	8.25	0.06	1.36	36.86	401.2	99.9	99.45
	Sarathi	46.22	144.2	0.1	0.14	90.12	417.6	4.44	1.9
	Sarathi+POD	11.74	27.38	0.09	0.12	40.6	333.0	2.2	2.1

Table 6. arXiv-based workload. Latency numbers in seconds.



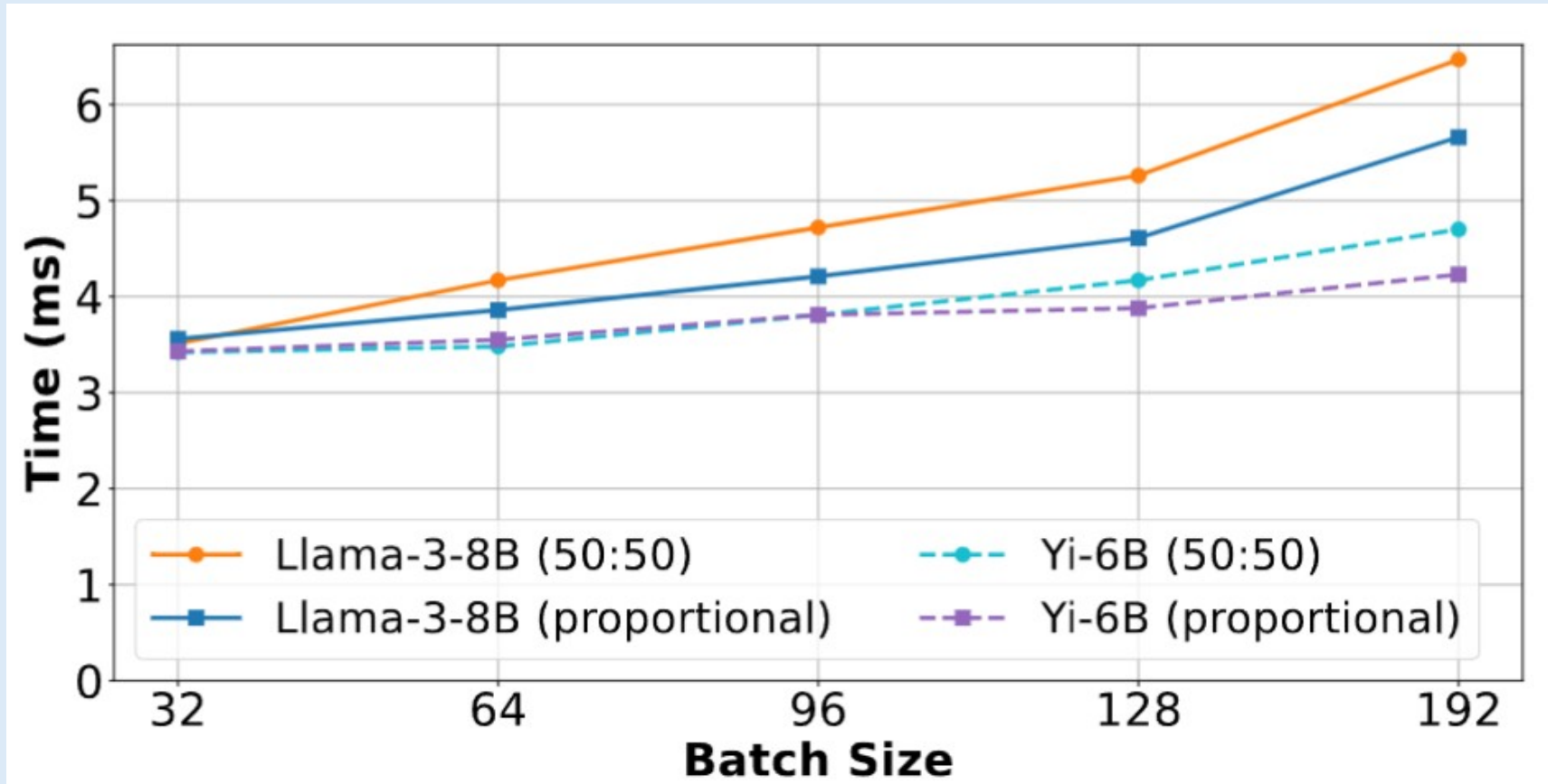
Steps to integrate POD-Attention

1. Convert prefill/decode kernels into generic device functions.
2. Remove references to CTA ID within kernel, instead pass as a parameter.

```
__global__ void flash_fwd_kernel(...params...)
to
__device__ void flash_fwd_func(...params..., int CTA_ID)
```

3. Use SM-aware CTA scheduling to pick whether to run prefill or decode and decide CTA ID to be passed.

Sensitivity: Scheduling style



8K context length,
tp-1